CS4330 Final Report

Urav Tanna, Vincent Chan

May 2026

Contents

1 Introduction

The ultimate goal of this project was to take a 5-stage pipelined MIPS processor and implement specific optimizations to improve overall performance while maintaining correctness. The processor was developed in C++ and implements a large subset of the MIPS ISA. We focused on three key optimizations: Out-of-Order (OOO) execution, superscalar execution, and a bimodal branch predictor. Together, these optimizations allowed us to achieve speedups of up to 6× on specific workloads.

The central question driving our evaluation was how scaling pipeline width impacts performance. Specifically, we wanted to answer: At around what width does performance plateau? Which workloads see the biggest gains from superscalar execution? And how does pipeline width interact with memory bottlenecks?

2 Architecture / Pipeline Overview

2.1 Pipeline Diagram

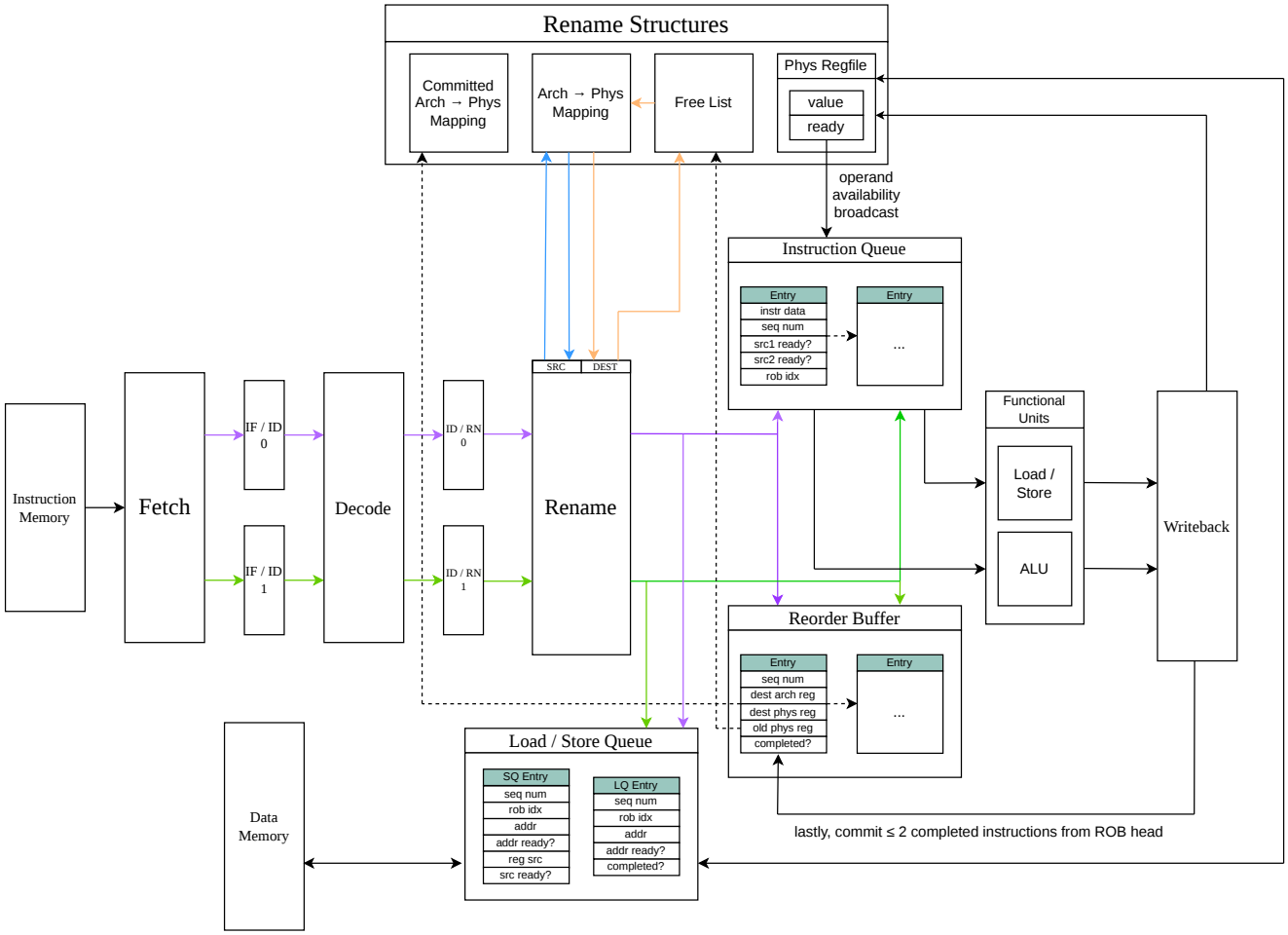

Figure 1: OOO Pipeline Diagram

Figure 1 depicts our OOO processor with `pipeline_width` set to 2. The number of pipeline registers and other components vary with `pipeline_width`, but we

limited the diagram to width 2 for readability.

2.2 Fetch and Decode

Fetch and Decode are largely unchanged from their in-order counterparts, with two key differences: a bimodal branch predictor is added in the Fetch stage, and both stages now process up to `pipeline_width` instructions per cycle instead of one.

2.3 Rename

The Rename stage handles both renaming and issue. For each instruction in the `ID_RN` pipeline registers: if the instruction writes to a register, a physical register is popped off the free list and assigned to the architectural destination in the register map. If the instruction is a load or store, it is added to the **load queue** or **store queue** respectively. The instruction is then added to both the **reorder buffer** and the **issue queue**.

2.4 Dispatch

In this stage we iterate through the **issue queue** looking for the oldest instruction whose source operands are ready. Once found, the instruction is dispatched to an available **functional unit**. This process repeats until all functional units are occupied or no ready instructions remain.

2.5 Execute

This stage is the most complex. All functional units in our implementation are capable of handling any instruction type, while unrealistic, this significantly simplified implementation. For **ALU operations**, source operands are read from the register file, the operation is performed, and the result is stored. For **branches**, the target address is calculated and the **Branch Target Buffer** is updated. If the actual target differs from the predicted target, a **squash** is initiated. Squashes walk the **reorder buffer**, **issue queue**, **non-blocking cache**, and **load/store queues**, eliminating all instructions with a greater sequence number than the mispredicted branch. If multiple mispredictions are resolved in the same cycle, the squash begins at the **oldest** branch. For **loads**, store-to-load forwarding is attempted first; on failure the L1 cache is probed; on a cache miss a **Miss Status Handling Register (MSHR)** is allocated if one is available. **Stores** compute their address and are handled in the Memory stage.

2.6 Memory

This stage handles instructions linked to the non-blocking cache. When load responses arrive, dependent instructions in the **issue queue** are woken up and

the value is written to the **physical register file**, and the corresponding **reorder buffer** entry is marked ready. Committed stores that could not write to memory during commit are also drained here.

2.7 Writeback

Functional units with completed results write back to the **physical register file** and wake dependent instructions. The corresponding **reorder buffer** entry is marked ready to commit.

2.8 Commit

Up to `pipeline_width` instructions are committed per cycle. Physical registers are reclaimed and loads are removed from the load queue. Stores remain in the store queue until their data has been written back to memory, but their **reorder buffer** entry is freed at commit.

3 Implementation Details

3.1 Simulator Constants

```
constexpr int NUM_PHYSICAL_REGISTERS = 96;
constexpr int NUM_ARCHITECTURAL_REGISTERS = 32;
constexpr int REORDER_BUFFER_SIZE = 64;
constexpr int PIPELINE_WIDTH = 8;
constexpr int INSTRUCTION_QUEUE_SIZE = 32;
constexpr int NUM_ALUS = 8;
constexpr int NUM_MSHRS = 16;
constexpr int LOAD_QUEUE_SIZE = 64;
constexpr int STORE_QUEUE_SIZE = 64;
constexpr int BTB_BITS = 8; //2 ^ size for btb arr length
```

These were the constants we settled on after some empirical tuning. MIPS ISA defines 32 architectural registers, and we wanted to ensure the free list never causes a bottleneck, so we set `NUM_PHYSICAL_REGISTERS` to `num_arch_regs + rob_size = 96`. The `REORDER_BUFFER_SIZE` required the most tuning; we found performance plateaued after around 64 entries. Similar reasoning applies to the `INSTRUCTION_QUEUE_SIZE`, `LOAD_QUEUE_SIZE`, and `STORE_QUEUE_SIZE`. While simulation imposes no hardware cost for larger structures, we wanted our parameters to be in the same order of magnitude as the processors that inspired this design, namely the MIPS R10000, rather than inflating them to artificially boost performance numbers. We saw very little gain from increasing `NUM_MSHRS` beyond 16, as there were rarely more than a few simultaneous in-flight cache misses. `PIPELINE_WIDTH` and `NUM_ALUS` were the parameters we varied most during testing. Increasing them to 8 generally yielded the greatest performance gains, though the returns diminished sharply depending on the workload.

3.2 Branch Prediction

The simulator uses a bimodal branch predictor to reduce the cost of control hazards. We initially attempted to implement a GShare predictor but had to scale back due to time constraints. Despite its simplicity, the predictor produced meaningful performance gains, as squashes are expensive in our pipeline. The `BranchPredictor` class stores an array of `BTBEntry` structs, each containing a `valid` bit, a `tag`, a `target` address, and a `bimodal_counter`. The `valid` bit indicates whether the entry is occupied, the `tag` verifies which branch the entry is tracking, `target` stores the predicted branch target, and `bimodal_counter` encodes the prediction state (weakly taken, strongly taken, etc.). The array is of size $2^{\texttt{BTB\_BITS}}$. During Fetch, we check whether the current instruction hits in the **BTB** and if so redirect `pc` to the stored target. During Execute, we verify whether the prediction was correct and update the predictor accordingly.

3.3 Instruction Queue

The **instruction queue** is a `vector` of `iq_instr` structs. We chose a `vector` because instruction queue indices are not referenced externally, so index shifting on removal is not a concern. Each `iq_instr` stores standard instruction metadata such as `pc`, `opcode`, and source registers, as well as fields for the `sequence_number`, `rob_index`, `lsq_index`, source operand ready flags, and the predicted taken/not-taken outcome. The two most important methods in the `InstructionQueue` class are `get_oldest_ready()`, which returns the ready instruction with the lowest sequence number, and `broadcast_ready(phys_reg)`, which iterates through the queue and marks source operands ready for any instruction dependent on the given physical register.

3.4 Functional Units

Each **functional unit** has a `ready` flag, an `iq_instr`, a `has_result` flag, a `result_instr`, and an integer `result`. The `result_instr` and `result` are read by the Writeback stage to update the **physical register file**. As mentioned earlier, every functional unit can handle any instruction type, and the meaning of `result` depends on the instruction context. Functional units are assigned instructions during the Dispatch stage.

3.5 Load / Store Queues

Both the **load queue** and **store queue** are circular buffers of `LoadEntry` and `StoreEntry` structs respectively. `LoadEntry` stores standard metadata along with the `address`, an `address_ready` flag, the destination physical register, and flags for whether the instruction has been issued or completed. It also stores whether the load is a byte or halfword to support partial store-to-load forwarding; this is not fully implemented in our current processor but we wanted to preserve the flexibility for future iterations. `StoreEntry` stores the same byte

range metadata along with the source register and data value, which is essential for store-to-load forwarding. Importantly, stores are not removed from the queue at commit; they remain until their data has been written back to the cache.

We use a conservative memory disambiguation policy: a load will not issue until all older stores in the store queue have computed their addresses. This ensures that store-to-load forwarding checks are always accurate. Implementing speculative load execution with recovery on mis-speculation was difficult for us in the given time frame, so we opted for this simpler but correct approach.

3.6 Reorder Buffer

The **reorder buffer** is a circular array, as several other data structures rely on `rob_index` for internal bookkeeping. The two most important fields in each entry are the `old_preg` associated with the instruction and the `completed` flag. Storing `old_preg` is essential for correctly restoring the free list during a squash. The `completed` flag signals that the instruction is ready to commit. The commit method pops up to `PIPELINE_WIDTH` instructions from the head of the buffer each cycle.

4 Performance Evaluation

4.1 Methodology

To evaluate the effectiveness of our optimizations, we benchmarked the processor across several workloads with varying levels of instruction-level parallelism and memory intensity, each selected to stress a different aspect of the design. Performance was measured by varying `pipeline_width` and `num_alus` while holding all other architectural parameters constant.

One important caveat is that our in-order pipelined processor, while passing the initial test suite, began failing the new test cases we introduced later in the project. Since our focus at that point was on getting the out-of-order processor correct, we did not have time to resolve these issues. As a compromise, we use a `pipeline_width = 1` out-of-order processor as our baseline. This is not a perfect stand-in for a true in-order processor since it still performs register renaming and uses a non-blocking cache, but it was the most reasonable baseline available given our time constraints. The performance gains we report therefore represent a lower bound on the true speedup over a conventional in-order pipeline.

4.2 High Instruction Level Parallelism Workloads

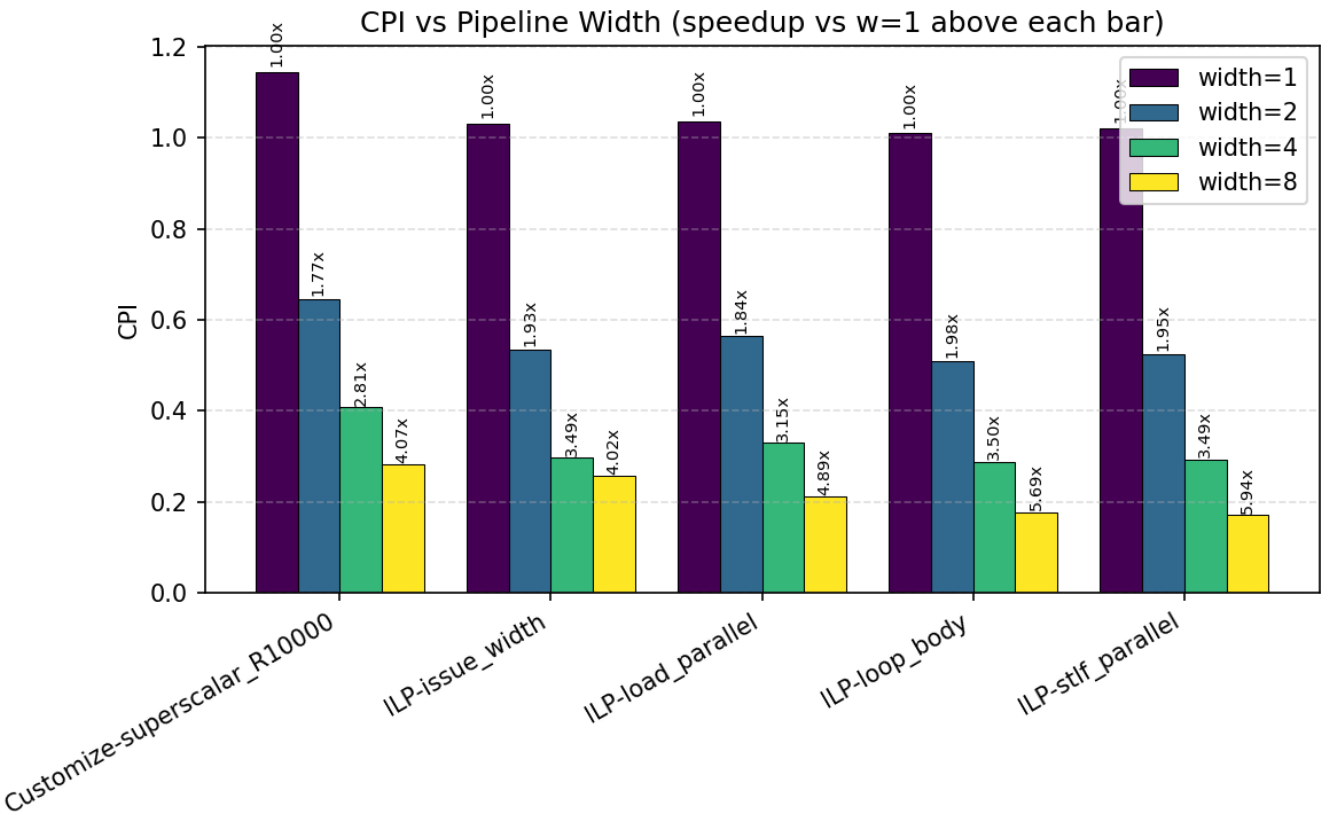

Figure 2: Benchmark Data for High ILP Workloads

These workloads were designed to maximize the benefits of superscalar execution. Each benchmark has a high degree of instruction-level parallelism, meaning many functional units are utilized simultaneously and multiple instructions are committed per cycle. Memory latency is low across these tests, which contributes to the strong CPI improvements we observe.

An interesting outlier is the `R10000` workload, which sees the least speedup. We believe this is due to the benchmark being relatively short; by the time the pipeline is fully saturated, the workload has nearly completed, limiting the window of peak utilization.

A particularly encouraging result is the `ILP-stlf_parallel` benchmark, which achieves a 6× speedup. This is most likely explained by the fact that loads are issued concurrently and are all forwarding directly from the **store**

queue, resulting in very low effective memory latency and a significant reduction in CPI.

Overall, superscalar execution performs extremely well on these workloads but hits a ceiling around a pipeline width of 4-8. This is consistent with Amdahl's Law: even in highly parallel workloads there is an irreducible serial fraction, and as pipeline width increases the parallel portion of execution is sped up while the serial portion remains fixed. The diminishing returns we observe past width 4 reflect this limit, where adding more functional units yields less and less marginal improvement.

4.3 Memory Heavy Workloads

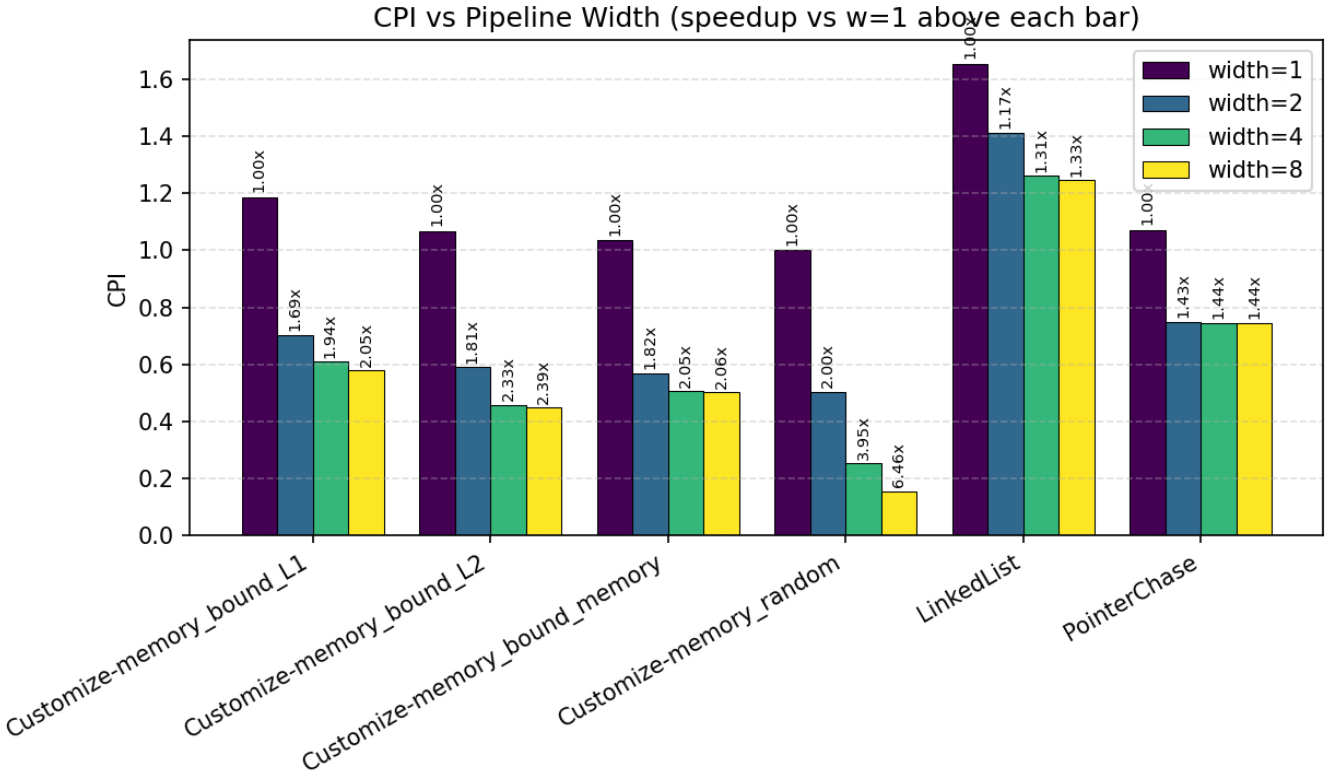

Figure 3: Benchmark Data for Memory Based Workloads

These workloads have higher memory dependency, so we would expect slower CPI improvement and less effective utilization of additional pipeline width. For the most part the results confirm this. Some test cases begin to plateau as

early as `pipeline_width = 2`, and while we do observe some speedup it flattens quickly.

`PointerChase` and `LinkedList` are the clearest examples of this behavior. The high number of dependent loads creates a chain of memory stalls that superscalar execution cannot hide, resulting in minimal CPI improvement regardless of pipeline width.

The most surprising result in this category was `Customize-memory_random`, which saw a sharper performance gain than expected. We are not entirely sure why, but our guess is that the workload contains a large number of independent loads that are able to utilize the **MSHRs** concurrently, effectively hiding memory latency and allowing the wider pipeline to do useful work.

Overall, memory-dependent workloads confirm the limits of superscalar execution. When the bottleneck is a chain of dependent memory accesses rather than instruction throughput, additional pipeline width provides little benefit, which is again consistent with Amdahl's Law.

4.4 Branch Predictor Performance

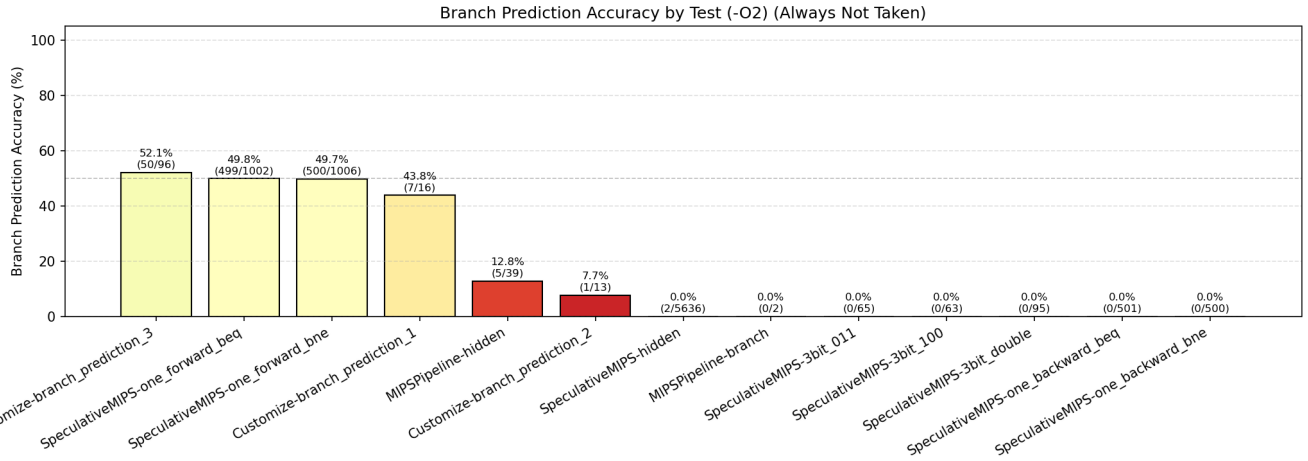

Figure 4: Branch Prediction Accuracy for Always Not Taken

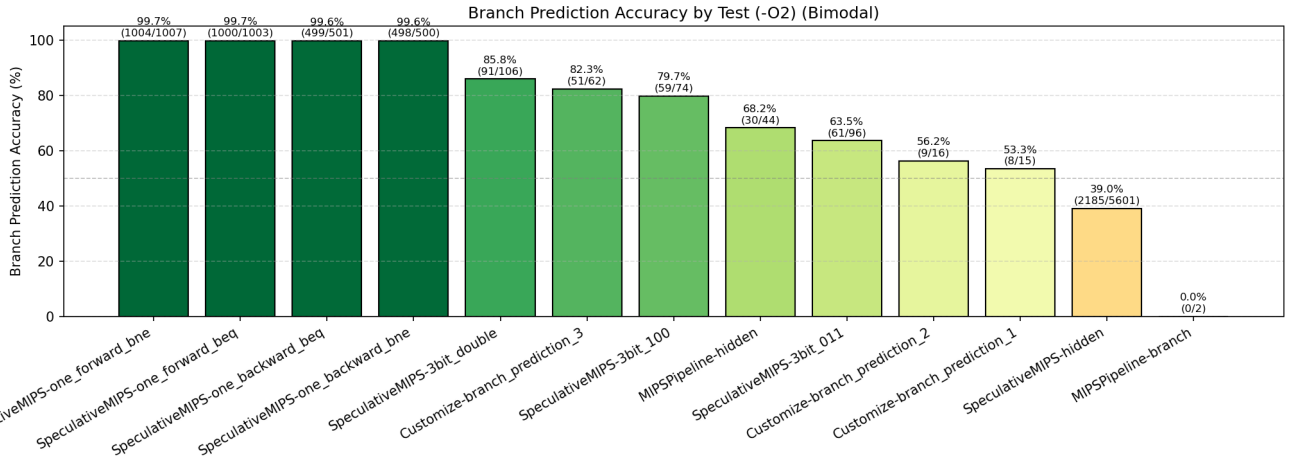

Figure 5: Branch Prediction Accuracy for Bimodal

Our initial processor used an always-not-taken branch policy, which resulted in very low prediction accuracy as shown in Figure 4. Replacing this with a bimodal predictor was one of the most impactful optimizations we made. This is unsurprising given that our pipeline is 8 stages deep, meaning a misprediction carries a high squash cost.

As Figure 5 shows, the bimodal predictor achieves close to 100% accuracy on several test cases. It is worth noting that these benchmarks are relatively simple, and more complex branching patterns would likely result in lower accuracy. For this workload suite however, the bimodal predictor performs very well and represents a meaningful improvement over the always-not-taken baseline.

5 Future Work

There are several directions we would like to explore in future iterations of this project.

5.1 Speculative Memory Disambiguation

We have a strong foundation with our **load queue** and **store queue** implementations and are not currently extracting the maximum value out of these structures. Allowing loads to issue speculatively ahead of older stores with unresolved addresses, with recovery on a mis-speculation, would meaningfully improve performance on memory-intensive workloads.

5.2 Branch Prediction

Our current bimodal predictor is simple and effective, but upgrading to a **GShare** predictor or a more sophisticated scheme like **TAGE** would reduce the frequency of costly squashes, particularly on workloads with irregular branch patterns.

5.3 Non-Blocking Cache Refactor

Our **non-blocking cache** implementation is functional but could benefit from a refactor. The current code is difficult to follow and improving its structure would make it easier to extend and reason about correctness.

5.4 In-Order Baseline

We would like to establish a cleaner baseline comparison by benchmarking against a true **in-order pipelined processor**. Our current width-1 out-of-order configuration still eliminates certain data hazards through register renaming and makes use of **MSHRs**, so while it approximates an in-order processor it is not a faithful one. A proper in-order baseline would let us isolate and quantify the performance contribution of out-of-order execution and non-blocking caches independently. We did have an in-order pipelined processor implementation, but as we developed new test cases late in the project we encountered correctness issues that we were unable to fully resolve in time. Getting a verified in-order baseline working and passing all test cases is something we would prioritize in a future iteration.

6 Artificial Intelligence Utilization

We utilized AI tools extensively throughout this project, primarily **Claude Code**. The most valuable aspect was having a tool that could hold the entire codebase in context and reason about it holistically, which was particularly useful for tracking down bugs and reviewing implementations for correctness.

We found the best results came from a clear division of labor: we made all major design decisions ourselves and used Claude to review the results, suggest modifications, and hunt for bugs. This kept us in control of the architecture and prevented the codebase from drifting in directions we did not fully understand.

Claude Code was especially effective once we had a solid testing infrastructure in place. Being able to autonomously make a change, run the test suite, inspect register output, and iterate without human intervention made debugging significantly faster. That said, it was not always efficient, there were several instances where a simple bug consumed an entire session to resolve, even with a paid plan.

Beyond debugging, Claude was useful for generating Python benchmarking scripts, automating graph generation, and writing MIPS assembly test cases,

which saved a considerable amount of time on the less intellectually interesting parts of the project.

Overall, we found the most effective way to use these tools is to treat them as a second set of eyes and a bug hunter rather than a designer. AI can contribute meaningfully to implementation, but designing the system yourself is important for maintaining understanding of the codebase and producing work you can actually reason about and defend.

7 Conclusion and Reflection

What initially seemed daunting became more manageable once we broke the project down into well-defined pieces. One of the core challenges of implementing an OOO processor is that the components are deeply interdependent; it is difficult to test any individual piece in isolation since everything operates in unison. The approach that worked best for us was to build in layers: data structures first, then individual pipeline stages, then full integration. This gave us a stable foundation at each step before introducing more complexity. We also found it useful to bring up instruction support incrementally, starting with ALU instructions only, then adding branches, and finally memory instructions. This allowed us to catch and fix fundamental bugs at each layer before moving on to more complex behavior.

If we were to start over, we would follow the same layered approach from the beginning. Having a clear build order and incremental testing infrastructure in place early was the single most important factor in keeping the project manageable. This project was one of the most complex and rewarding we have taken on during our time at UVA. Grinding through bugs and optimizing the processor was a slog at points, but we learned a great deal throughout the process.